

NumPy- Advanced Array Manipulation

Advanced manipulation of NumPy arrays involves a variety of techniques. They can be classified into the following types –

- ✓ **Changing Shape**
- ✓ **Transpose Operation**
- ✓ **Joining Array**
- ✓ **Splitting Arrays**
- ✓ **Adding / Removing Elements**

Changing Shape:-

In NumPy, changing the shape of an array can be achieved using several functions. The most common ones are **reshape**, **flat**, **flatten** and **ravel**.

1) **reshape()**

The reshape function allows you to change the shape of an array without modifying its data.

```
import numpy as np
arr = np.arange(12)
print(arr)
rshape = arr.reshape((3, 4))
print(rshape)
#OR
rshape2=np.reshape(arr,(4,3))
print(rshape2)
```

2) **flat()**

This function returns a 1-D iterator over the array. It behaves similar to Python's built-in iterator.

```
import numpy as np
a = np.arange(1,9).reshape(2,4)
print("The original array:")
print(a)
print("After applying the flat function:")
# returns element corresponding to index in flattened array
print(a.flat[6])
```

3) **flatten()**

This function returns a copy of an array collapsed into one dimension. The function takes the following parameters.

'C'— row major (default. 'F': column major 'A': flatten in column-major order, if a is Fortran contiguous in memory, row-major order otherwise 'K': flatten a in the order the elements occur in the memory

```
import numpy as np
a = np.arange(1,9).reshape(2,4)
print ('The original array is:')
print (a)
# default is column-major
print ('The flattened array is:')
print (a.flatten())
print ('The flattened array in F-style ordering:')
print (a.flatten(order = 'C'))
print (a.flatten(order = 'F'))
print (a.flatten(order = 'A'))
print (a.flatten(order = 'K'))
```

4) **ravel()**

This function returns a flattened one-dimensional array. A copy is made only if needed. The returned array will have the same type as that of the input array.

'C': row major (default. 'F': column major 'A': flatten in column-major order, if a is Fortran contiguous in memory, row-major order otherwise 'K': flatten a in the order the elements occur in the memory

```
import numpy as np
a = np.arange(8).reshape(2,4)
print ('The original array is:')
print (a)
print ('After applying ravel function:')
print (a.ravel())
print ('Applying ravel function in F-style ordering:')
print (a.ravel(order = 'F'))
```

Transpose Operation:-

The transpose operation in NumPy rearranges the dimensions of an array. For a 2D array, the transpose is obtained by swapping rows with columns. For higher-dimensional arrays, the transpose operation permutes the axes according to the specified order.

1) **transpose()**

Permutes the dimensions of an array (row into column & column into row)

```
import numpy as np
a = np.arange(1,16).reshape(3,5)
print('The original array is:')
print(a)
print('The transposed array is:')
print(np.transpose(a))
```

2) ndarray.T

This function belongs to ndarray class. It behaves similar to numpy.transpose.

```
import numpy as np
a = np.arange(1,16).reshape(3,5)
print('The original array is:')
print(a)
print('The transposed array is:')
print(a.T)
```

3) rollaxis()

This function rolls the specified axis backwards, until it lies in a specified position. The function takes three parameters.

numpy.rollaxis(arr, axis, start)

```
import numpy as np
a = np.arange(8).reshape(2,4)
print ('The original array:')
print(a)
# to roll axis-2 to axis-0 (along width to along depth)
print ('After applying rollaxis function:')
print (np.rollaxis(a,1))
# to roll axis 0 to 1 (along width to height)
print( 'After applying rollaxis function:' )
print (np.rollaxis(a,1,1))
```

4) swapaxes()

This function interchanges the two axes of an array. For NumPy versions after 1.10, a view of the swapped array is returned. The function takes the following parameters.

numpy.swapaxes(arr, axis1, axis2)

```
import numpy as np
a = np.arange(8).reshape(2,4)
print ('The original array:')
print(a)
# to roll axis-2 to axis-0 (along width to along depth)
print ('After applying rollaxis function:')
print (np.swapaxes(a,1,0))
# to roll axis 0 to 1 (along width to height)
print( 'After applying rollaxis function:' )
print (np.swapaxes(a,1,1))
```

Joining Array:-

Joining arrays in NumPy, you can use methods like `numpy.concatenate`, `numpy.vstack`, `numpy.hstack`, and `numpy.stack`. Here's a function that incorporates these methods:

1) `concatenate()`

Concatenation refers to joining. This function is used to join two or more arrays of the same shape along a specified axis. The function takes the following parameters.

`numpy.concatenate((a1, a2, ...), axis)`

```
import numpy as np
a = np.array([[1,2],[3,4]])
print("First Array:-")
print(a)
b = np.array([[5,6],[7,8]])
print("Second Array:-")
print(b)
# both the arrays are of same dimensions
print('Joining the two arrays along axis 0:')
print(np.concatenate((a,b)))
print('Joining the two arrays along axis 1:')
print(np.concatenate((a,b),axis = 1))
```

2) `stack()`

This function joins the sequence of arrays along a new axis. This function has been added since NumPy version 1.10.0. Following parameters need to be provided.

```
numpy.stack(arrays, axis)
import numpy as np
a = np.array([[1,2],[3,4]])
print ('First Array:')
print (a)
b = np.array([[5,6],[7,8]])
print ('Second Array:')
print (b)
print ('Stack the two arrays along axis 0:')
print (np.stack((a,b),0))
print ('Stack the two arrays along axis 1:')
print (np.stack((a,b),1))
```

3) `hstack()`

Stacks arrays in sequence horizontally (column wise)

```

import numpy as np
a = np.array([[1,2],[3,4]])
print ('First array:')
print (a)
b = np.array([[5,6],[7,8]])
print ('Second array:')
print (b)
print ('Horizontal stacking:')
c = np.hstack((a,b))
print (c)

```

4) **vstack()**

Stacks arrays in sequence vertically (row wise)

```

import numpy as np
a = np.array([[1,2],[3,4]])
print ('First array:')
print (a)
b = np.array([[5,6],[7,8]])
print ('Second array:')
print (b)
print ('Vertical stacking:')
c = np.vstack((a,b))
print (c)

```

Splitting Arrays:-

This function divides the array into subarrays along a specified axis. The function takes three parameters.

`numpy.split(ary, indices_or_sections, axis)`

1) **split()**

Splits an array into multiple sub-arrays

```

import numpy as np
a = np.arange(9)
print ('First array:')
print (a)
print ('Split the array in 3 equal-sized subarrays:')
b = np.split(a,3)
print (b)
print ('Split the array at positions indicated in 1-D array:')
b = np.split(a,[4,7])
print (b)

```

2) **hsplit()**

Splits an array into multiple sub-arrays horizontally (column-wise)

The `numpy.hsplit` is a special case of `split()` function where axis is 1 indicating a horizontal split regardless of the dimension of the input array.

```
import numpy as np
a = np.arange(16).reshape(4,4)
print ('First array:')
print (a)
print ('Horizontal splitting:')
b = np.hsplit(a,2)
print (b)
```

3) **vsplit()**

Splits an array into multiple sub-arrays vertically (row-wise)

`numpy.vsplit` is a special case of `split()` function where axis is 1 indicating a vertical split regardless of the dimension of the input array.

```
import numpy as np
a = np.arange(16).reshape(4,4)
print ('First array:')
print (a)
print ('Horizontal splitting:')
b = np.hsplit(a,2)
print (b)
```

Adding / Removing Element:-

1) **resize()**

This function returns a new array with the specified size. If the new size is greater than the original, the repeated copies of entries in the original are contained.

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print ('Original array:')
print(a)
print("The shape of Original array:")
print (a.shape)
# Observe that first row of a is repeated in b since size is bigger
print('After Resize the array:')
b = np.resize(a,(5,5))
print(b)
print("The shape of Resize array:")
print (b.shape)
```

2) **append()**

This function adds values at the end of an input array. The append operation is not inplace, a new array is allocated. Also the dimensions of the input arrays must match otherwise ValueError will be generated.

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print ('First array:')
print(a)
print ('Append elements to array:')
b=np.append(a, [7,8,9])
print(b)
print ('Append elements along axis 0:')
c= np.append(a, [[7,8,9]],axis = 0)
print(c)
print ('Append elements along axis 1:')
d=np.append(a, [[5,5,5],[7,8,9]],axis = 1)
print(d)
```

3) **insert()**

This function inserts values in the input array along the given axis and before the given index. If the type of values is converted to be inserted, it is different from the input array.

```
import numpy as np
a = np.array([[1,2],[3,4],[5,6]])
print('First array:')
print (a)
print('Axis parameter not passed. The input array is flattened before insertion.')
b=np.insert(a,3,[11,12])
print(b)
print ('Axis parameter passed. The values array is broadcast to match input array.')
print ('Broadcast along axis 0:')
c=np.insert(a,1,[11],axis = 0)
print(c)
print ('Broadcast along axis 1:')
d=np.insert(a,1,11,axis = 1)
print(d)
```

delete():- This function returns a new array with the specified subarray deleted from the input array. As in case of insert() function, if the axis parameter is not used, the input array is flattened. `Numpy.delete(arr, obj, axis)`

```

import numpy as np
a = np.arange(1,13).reshape(3,4)
print ('First array:')
print(a)
print ('Array flattened before delete operation as axis not used:')
b=np.delete(a,5)
print(b)
print ('Column 2 deleted:')
c=np.delete(a,1,axis = 1)
print(c)
print ('A slice containing alternate values from array deleted:')
a = np.array([1,2,3,4,5,6,7,8,9,10])
d=np.delete(a, np.s_[:,2])
print(d)

```

4) **unique()**

This function returns an array of unique elements in the input array. The function can be able to return a tuple of array of unique vales and an array of associated indices.

```

import numpy as np
a = np.array([5,2,6,2,7,5,6,8,2,9])
print ('First array:')
print(a)
print ('Unique values of first array:')
u = np.unique(a)
print(u)
print('Unique array and Indices array:')
u,indices = np.unique(a, return_index = True)
print (indices)
print(u)
print ('Return the count of repetitions of unique elements:')
u,indices = np.unique(a,return_counts = True)
print (u)
print (indices)

```


Assignment

- 1) Write a NumPy program that performs element-wise addition using broadcasting.
- 2) Write a NumPy program that creates a 2D array x of shape (3, 4) and a 1D array y of shape (4,). Subtract y from each row of x using broadcasting.
- 3) Write a NumPy program that multiplies a 3D array of shape (2, 3, 4) by a 1D array of shape (4,) using broadcasting.
- 4) Write a NumPy program that adds a scalar value to a 2D array of shape (5, 5) using broadcasting.
- 5) Write a NumPy program to Replace all even numbers in a 1D array with their negative.
- 6) Write a NumPy program to find the indices of non-zero elements from [1,2,0,0,4,0].
- 7) Write a NumPy program to find the index of the maximum value in a 1D array.
- 8) Write a NumPy program to find the mean of each row in a 2D array.
- 9) Write a NumPy program to find the unique values and their counts in a 1D array.
- 10) Write a NumPy program to find the unique values and their index in a 1D array.